



VAPT TASK 4

Theoretical Knowledge

1. Advanced Vulnerability Exploitation

A multi-stage attack in cybersecurity is a complex, multi-step intrusion designed to evade detection by breaking down a malicious operation into sequential phases, where each stage builds on the last to achieve the attacker's goal, like stealing data or deploying ransomware, often starting with a seemingly harmless action (like clicking a link) to gain initial access and then downloading more sophisticated tools or spreading laterally. These attacks are hard to spot because individual steps look benign, making it difficult for security systems to connect the dots and identify the full attack chain. In these kinds of hacks, the attackers don't go after their victims' networks directly. Instead, they penetrate the system of a third-party supplier with access to their targets' network assets.

Custom Exploit Development: Crafting a heap spray for browser exploits involves using client-side scripting (typically JavaScript) to intentionally fill the browser's memory heap with numerous copies of an attacker's malicious code (shellcode). The goal is to ensure that when a separate memory corruption vulnerability is triggered, the program's execution flow is redirected to a predictable address within this sprayed memory area, leading to the execution of the shellcode.

Bypassing Defense: Evasion of ASLR (Address Space Layout Randomization) and DEP (Data Execution Prevention) involves attackers using Return-Oriented Programming (ROP) to chain existing code snippets (gadgets) to bypass DEP, while ASLR bypasses rely on information leaks, predictable mappings, or JIT spraying to find gadget addresses, often using techniques like polymorphic payloads to dynamically alter code and evade detection by Web Application Firewalls (WAFs) through obfuscation and changing signatures.

Bypassing ASLR & DEP

- **Return-Oriented Programming (ROP):** Instead of injecting shellcode, ROP chains small instruction sequences (gadgets) ending in ret instructions, found within the program's own memory, to execute malicious logic.
- **DEP Bypass:** ROP inherently bypasses DEP (NX bit) because it executes existing, legitimate code, not injected data.
- **ASLR Bypass:** Attackers need to find the base addresses of these gadgets. Techniques include:
- **Information Leaks:** Exploiting vulnerabilities to read memory addresses, revealing library locations.



- **JIT Spraying:** In browsers, using Just-In-Time compilers to generate large chunks of executable code (shellcode) in predictable memory areas.
- **Predictable Mappings:** Relying on third-party DLLs or JIT-generated code having consistent base addresses.

EternalBlue: Case Study

EternalBlue is a Microsoft exploit which was used by the NSA in intelligence gathering operations. The exploit, officially named MS17-010 by Microsoft — gave the US National Security Agency (NSA) backend access to devices running Windows operating systems like Windows XP and Windows 7. After being aware of a weakness in Microsoft's SMBv1 (Server Message Block version 1) file-sharing protocol for five years, the NSA finally informed Microsoft of its existence. However, by the time they did, it had been leaked by a notorious hacking collective known as Shadow Brokers. The leak put millions of users at risk and the entire incident underlined the threats posed by the NSA's development and maintenance of software backdoors.[2]

How does EternalBlue work? [2]

The EternalBlue exploit worked by taking advantage of the unsecure SMBv1 protocol. This protocol allowed Microsoft devices to communicate with other Microsoft systems — carrying out file and print services, for example — but was vulnerable to manipulation.

To carry out the EternalBlue exploit, attackers just needed to send a malicious SMBv1 data packet to a Windows server that had the vulnerability. The packet would contain a payload of malware, which could then be rapidly disseminated to other devices installed with the vulnerable Microsoft software.

Once the Shadow Brokers leaked the exploit in 2017, hackers took advantage of the vulnerability to carry out devastating attacks and spread massive amounts of malware. Two notable incidents exemplify the effects of the vulnerability.

Heap Spraying

Heap spraying is a technique used in exploits to facilitate arbitrary code execution. It is a classic exploitation technique where an attacker deliberately fills the heap (a region of memory used by applications) with copies of their malicious code. The part of the source code of an exploit that implements this technique is called a **heap spray**. [4] In general, code that *sprays the heap* attempts to put a certain sequence of bytes at a predetermined location in the memory of a target process by having it allocate (large) blocks on the process's heap and fill the bytes in these blocks with the right values. [5]

A heap spray does not actually exploit any security issues but it can be used to make a vulnerability easier to exploit. A heap spray by itself cannot be used to break any security boundaries: a separate security issue is needed. [4]



2. API Security Testing

API Security Top 10 2023

- **API1:2023 - Broken Object Level Authorization**

APIs tend to expose endpoints that handle object identifiers, creating a wide attack surface of Object Level Access Control issues. Object level authorization checks should be considered in every function that accesses a data source using an ID from the user. .

- **API2:2023 - Broken Authentication**

Authentication mechanisms are often implemented incorrectly, allowing attackers to compromise authentication tokens or to exploit implementation flaws to assume other user's identities temporarily or permanently. Compromising a system's ability to identify the client/user, compromises API security overall. .

- **API3:2023 - Broken Object Property Level Authorization**

This category combines API3:2019 Excessive Data Exposure and API6:2019 - Mass Assignment, focusing on the root cause: the lack of or improper authorization validation at the object property level. This leads to information exposure or manipulation by unauthorized parties. .

- **API4:2023 - Unrestricted Resource Consumption**

Satisfying API requests requires resources such as network bandwidth, CPU, memory, and storage. Other resources such as emails/SMS/phone calls or biometrics validation are made available by service providers via API integrations, and paid for per request. Successful attacks can lead to Denial of Service or an increase of operational costs. .

- **API5:2023 - Broken Function Level Authorization**

Complex access control policies with different hierarchies, groups, and roles, and an unclear separation between administrative and regular functions, tend to lead to authorization flaws. By exploiting these issues, attackers can gain access to other users' resources and/or administrative functions. .

- **API6:2023 - Unrestricted Access to Sensitive Business Flows**

APIs vulnerable to this risk expose a business flow - such as buying a ticket, or posting a comment - without compensating for how the functionality could harm the business if used excessively in an automated manner. This doesn't necessarily come from implementation bugs.

- **API7:2023 - Server Side Request Forgery**

Server-Side Request Forgery (SSRF) flaws can occur when an API is fetching a remote resource without validating the user-supplied URI. This enables an attacker to coerce the



application to send a crafted request to an unexpected destination, even when protected by a firewall or a VPN. .

- **API8:2023 - Security Misconfiguration**

APIs and the systems supporting them typically contain complex configurations, meant to make the APIs more customizable. Software and DevOps engineers can miss these configurations, or don't follow security best practices when it comes to configuration, opening the door for different types of attacks

- **API9:2023 - Improper Inventory Management**

APIs tend to expose more endpoints than traditional web applications, making proper and updated documentation highly important. A proper inventory of hosts and deployed API versions also are important to mitigate issues such as deprecated API versions and exposed debug endpoints. .

- **API10:2023 - Unsafe Consumption of APIs**

Developers tend to trust data received from third-party APIs more than user input, and so tend to adopt weaker security standards. In order to compromise APIs, attackers go after integrated third-party services instead of trying to compromise the target API directly. .

Bypassing rate limits and injecting payloads

Bypassing rate limits and injecting payloads are techniques used in web application security testing to circumvent security measures and exploit underlying vulnerabilities.

- **Rate Limits:** Restrictions on how many requests a user/IP can make in a given time (e.g., 5 login attempts per minute) to prevent brute-force attacks.
- **Rate Limit Bypass:** Techniques to circumvent these restrictions to make excessive requests without getting blocked.
- **Payload Injection:** Inserting specifically crafted data (malicious code, special characters, or altered parameters) into web inputs to manipulate application logic.

3. Privilege Escalation and Persistence

Kernel Exploits

A kernel exploit is a type of cyberattack that targets vulnerabilities within the **kernel**—the core component of an operating system (OS) that has unrestricted access to hardware, memory, and CPU resources. Because the kernel operates at the highest privilege level (often referred to as Ring 0 or Supervisor Mode), a successful kernel exploit allows an attacker to bypass security controls, gain absolute control over the system, and escalate privileges from a standard user to root or Administrator.

Common Kernel Vulnerabilities



Kernel exploits leverage various memory corruption bugs to manipulate the kernel's behaviour:

- **Use-After-Free (UAF):** The kernel continues to use a pointer to memory that has already been freed, allowing an attacker to reuse that memory for malicious purposes.
- **Buffer Overflows (Stack/Heap):** Writing more data to a buffer than it can hold, allowing an attacker to overwrite adjacent memory, including critical data structures.
- **Race Conditions:** Exploiting the timing difference between checking a resource and using it, such as in the **Dirty COW** (CVE-2016-5195) vulnerability.
- **Information Leaks (Infoleaks):** Exposing memory contents (like stack addresses or canary values) that are used to bypass defenses like KASLR (Kernel Address Space Layout Randomization).

Examples of Famous Kernel Exploits

- **Dirty COW (CVE-2016-5195):** A race condition in the Linux memory subsystem that allowed privilege escalation.
- **Dirty Pipe (CVE-2022-0847):** A flaw allowing unprivileged users to overwrite data in read-only files.
- **EternalBlue (CVE-2017-0144):** A Windows SMBv1 vulnerability that enabled remote code execution in the kernel, used by WannaCry ransomware.
- **SLUBStick (2024):** A technique that exploits Linux kernel heap vulnerabilities to achieve root privileges.

Detection and Mitigation

Kernel exploits are dangerous because they can hide from security software.

- **Patching:** Keeping the kernel updated is the most crucial defense, as patches fix known vulnerabilities.
- **Kernel Hardening:** Techniques like KASLR, supervisor mode access prevention (SMAP), and Control Flow Integrity (CFI) make exploitation difficult.
- **Unsigned Driver Blocking:** Modern systems (like Windows) prevent unsigned drivers from loading to stop malicious code from running in the kernel space.

4. Network Protocol Attacks

Protocol attack vectors, which focus on exploiting weaknesses in network communication rules rather than just software vulnerabilities, are a significant focus in network security analysis. These attacks often target resource depletion or connection manipulation, frequently leveraging tools like Wireshark and Nmap to identify and exploit vulnerabilities in the TCP/IP suite.

ARP spoofing, DNS poisoning, and SSL stripping are three distinct techniques used for data interception, typically as part of a larger Man-in-the-Middle (MitM) attack. These



methods exploit vulnerabilities in network protocols and communication security to allow an attacker to intercept, read, or modify data in transit.

ARP Spoofing

- **Mechanism:** The Address Resolution Protocol (ARP) is used by devices on a local network to map IP addresses to physical MAC addresses. ARP spoofing (or ARP poisoning) involves an attacker sending fake ARP messages across the network.
- **How it Works:** The attacker tricks the victim's device and the network router into believing that their MAC address is associated with the router's IP address (from the victim's perspective) and the victim's IP address (from the router's perspective). As a result, all network traffic between the victim and the router is rerouted through the attacker's machine.
- **Data Interception:** Once the traffic flows through the attacker, they can intercept sensitive data like usernames, passwords, and session cookies if the communication is unencrypted.

DNS Poisoning

- **Mechanism:** DNS poisoning (or DNS spoofing) involves corrupting the Domain Name System (DNS) cache data on a victim's machine or a DNS server [2, 1].
- **How it Works:** The attacker provides fraudulent DNS records, redirecting the victim's request for a legitimate website (e.g., www.bank.com) to a malicious IP address controlled by the attacker.
- **Data Interception:** The victim's browser is directed to a fake website that appears identical to the real one. Any credentials or personal information entered on this malicious site are captured by the attacker. This can also be a component of a MitM attack after initial interception via ARP spoofing.

SSL Stripping

- **Mechanism:** SSL stripping (also known as HTTPS downgrading) is an attack that specifically targets secure HTTPS connections.
- **How it Works:** The attacker typically first positions themselves in the network via an ARP spoofing attack. When the victim attempts to visit a secure website (HTTPS), the attacker intercepts the initial request and communicates with the legitimate website over HTTPS, while communicating with the victim over an insecure HTTP connection. The victim's browser displays a standard HTTP connection, usually without any security warnings they might typically expect.
- **Data Interception:** The attacker decrypts all traffic coming from the secure website and re-encrypts it (or leaves it as plain text) for the victim. The attacker can then read all data, including credentials and personal information, in plain text as it passes through their machine.

Common Defense Mechanisms include using a Virtual Private Network (VPN) on untrusted networks, ensuring the use of HTTPS with a valid certificate, and implementing network security monitoring tools that can detect unusual ARP activity



5. Mobile Application Penetration Testing

OWASP Top 10 Mobile Risks – 2014 [1]

- M1: Weak Server Side Controls
- M2: Insecure Data Storage
- M3: Insufficient Transport Layer Protection
- M4: Unintended Data Leakage
- M5: Poor Authorization and Authentication
- M6: Broken Cryptography
- M7: Client Side Injection
- M8: Security Decisions Via Untrusted Inputs
- M9: Improper Session Handling
- M10: Lack of Binary Protections

Secure storage, code obfuscation, and runtime checks

Implementing a combination of **secure storage**, **code obfuscation**, and **runtime checks** is a, comprehensive, and widely recommended strategy to protect mobile and desktop applications against reverse engineering, intellectual property theft, and tampering. This layered approach turns a standard application into a "hardened" one, significantly increasing the time and resources required for an attacker to analyze or modify the code.

1. Secure Storage

This component ensures that sensitive data (API keys, user credentials, tokens, local databases) is not stored in plain text.

- **iOS Keychain & Android Keystore:** The industry standard for securely storing small pieces of data. These systems use hardware-backed, encrypted containers to protect cryptographic keys and sensitive information.
- **Encrypted Preferences/Databases:** For larger data, using encrypted SQL databases or encrypted shared preferences ensures that data at rest is unreadable, even if the device is rooted or compromised.
- **White-box Cryptography:** A more advanced form of data protection that embeds cryptographic keys within the obfuscated code, making it difficult for an attacker to extract them even while the application is running.

2. Code Obfuscation

Code obfuscation transforms the application's source code or bytecode into a version that is functionally identical to the original but extremely difficult for humans and AI to read.

- **Renaming (Layout Obfuscation):** Replaces meaningful names for classes, methods, and variables with meaningless, scrambled ones (e.g., calculateTotal becomes a1B2C3).
- **Control Flow Obfuscation:** Modifies the logical flow of the code, making it hard to follow. This includes flattening the code (replacing loops/conditionals with a switch-based dispatcher).



- **String Encryption:** Encrypts strings at rest and decrypts them only at runtime, preventing attackers from finding API endpoints, hardcoded keys, or sensitive text.
- **Instruction Substitution & Junk Code:** Replaces simple instructions with complex ones and inserts dead code (code that does nothing) to confuse analysis tools.

3. Runtime Checks (RASP)

Runtime Application Self-Protection (RASP) monitors the application while it is running, reacting in real-time to tampering attempts.

- **Root/Jailbreak Detection:** Detects if the device is running a compromised operating system, which would allow an attacker to bypass standard security sandboxes.
- **Anti-Debugging:** Detects if a debugger is attached to the process and shuts down the application or alerts the server to prevent inspection of memory.
- **Anti-Hooking:** Prevents frameworks like Frida or Xposed from hooking into the application's functions to change their behavior.
- **Code Integrity Checks (Checksumming):** Periodically verifies that the application binary has not been modified (repackaged or tempered with).

The "Layered" Security Approach

- **Why it's necessary:** Obfuscation alone is not a silver bullet; it acts as a "speed bump" rather than a wall, providing little defense against runtime attacks like dynamic instrumentation.
- **Synergy:** When combined, these three techniques protect the application at rest (secure storage), during analysis (obfuscation), and during execution (runtime checks), creating a robust, multi-layered defense.
- **Tools:** Common tools for implementing these techniques include **DexGuard/iXGuard** (Guardsquare), **Jscrambler** (JavaScript), **ProGuard/R8** (Android), and **Obfuscator-LLVM** (native code).



Figure 1:OWASP TOP 10 MOBILE VULNERABILITIES



References

- [1] <https://owasp.org/www-project-mobile-top-10/>
- [2] <https://nordvpn.com/blog/what-is-eternalblue/?srsId=AfmBOor4rdFI9DekJJawxACEMA4R2mv2Up2zSNatl6zSv7vZiOgLYgh8>
- [3] <https://owasp.org/www-project-api-security/>
- [4] https://en.wikipedia.org/wiki/Heap_spraying
- [5] <https://www.huntress.com/cybersecurity-101/topic/what-is-heap-spraying>